Jonathan Brugh

Enhancement Two Narrative

08/08/2026

Overview

Enhancement Two is the second and deeper revision of the course planner, and it is squarely aimed at the data structure itself rather than the surrounding program. Where Enhancement One focused on hardening input handling and cleaning up the interface around the binary search tree (BST), Enhancement Two goes into the tree. It implements the delete operation that both prior versions only declared, corrects a structural flaw in how the tree handled duplicate keys, changes how the tree manages its own memory, and improves how course data moves through the recursive insertion algorithm. These are the kinds of changes that demonstrate fluency with data structure design and algorithmic correctness, not just program-level polish.

Implementing the Delete Operation the Prior Versions Only Promised

Both the original artifact and Enhancement One declare a removeNode() method in the BinarySearchTree class, and both leave it unimplemented. Deleting a course was simply not possible in either version, and the class comment in Enhancement One still flags this as unfinished work. Enhancement Two closes that gap with a complete, correct implementation of the standard recursive BST deletion algorithm.

The algorithm handles all three cases a BST deletion must account for. If the node being removed has no left child, its right subtree is promoted in its place; if it has no right child, its left subtree is promoted. If it has two children, the algorithm cannot simply remove the node without

breaking the ordering invariant, so it locates the node's in-order successor -- the leftmost node of its right subtree, which is found by a new getSuccessor() helper -- copies that successor's data into the node being deleted, and then recursively removes the successor from its original position, where it is guaranteed to have at most one child. This is the textbook approach to BST deletion, and implementing it correctly requires understanding why each case exists: removing a two-child node incorrectly is one of the most common ways a BST implementation silently corrupts itself, because it is easy to detach a subtree without checking whether the tree's ordering property still holds for every remaining node.

DeleteCourse() at the public interface simply hands the current root to removeNode() and reassigns the tree's root to whatever removeNode() returns, which is idiomatic for a recursive structure expressed with smart pointers: the function returns ownership of the subtree it was given, whether that subtree is unchanged, restructured, or now empty.

Fixing a Structural Flaw: Duplicate Keys

A BST is only a valid search structure if each key appears at most once and every node's position is consistent with a single, well-defined comparison rule. Enhancement One's addNode() does not check for an existing match. It only asks whether the new courseId is less than the current node's courseId, and anything not less than falls into the right-subtree branch. If a CSV file or a user re-entered a courseId that already existed in the tree, that course would be inserted as a brand-new node rather than replacing the old one, leaving two nodes in the tree with the same key. From that point on, Search() and InOrder() behave unpredictably for that courseId, since the search algorithm's correctness depends on there being exactly one place in the tree where a given key can live.

Enhancement Two corrects this at the root of the insertion algorithm: addNode() first checks whether the incoming course's ID exactly matches the current node's ID, and if so, overwrites that node's data in place and returns immediately, without descending further. This single check restores the BST's key-uniqueness invariant and is also what makes the new update-or-insert workflow in the menu, case 4, safe to use. Re-entering an existing courseId now reliably updates that one node instead of quietly duplicating it.

Guaranteeing a Consistent Ordering Key

A related, easy-to-miss correctness issue is that a BST's comparisons are only meaningful if the key used for comparison is normalized the same way every time it is produced. Enhancement One stores courseId exactly as it is typed or read from a CSV, so 'CS300' and 'cs300' are treated as different strings with a different sort position, even though they represent the same course to a person using the program. Enhancement Two runs every courseId through a new ToUpper() helper before it is inserted, searched, updated, or deleted, so the same logical course always produces the same key no matter how a user or a CSV file happens to capitalize it. This is a small function, but it is doing real structural work, and it is what makes the tree's ordering property actually correspond to the real-world identity of a course.

Rethinking Memory Ownership With Smart Pointers

Enhancement One's Node still uses raw Node* left and right pointers, which means the tree is responsible for manually walking itself and calling delete on every node in deleteBST(), invoked from the class destructor. Enhancement Two replaces those raw pointers with std::unique_ptr<Node>. Ownership of a subtree now belongs to exactly one pointer at a time, and that pointer's destructor automatically and recursively destroys everything beneath it when

the tree goes out of scope. deleteBST() and the explicit virtual destructor disappear entirely, because the language's own object lifetime rules now do that work correctly and safely.

This is not just a style change. The recursive removeNode() function depends on unique_ptr's move semantics to detach and reattach subtrees safely: std::move(node->left) transfers ownership of an entire subtree into a recursive call without copying it, and the function's return value hands ownership back to the caller. Using a smart pointer here is what allows the deletion algorithm to restructure the tree, promoting a child and splicing in a successor, without ever risking a dangling pointer, a double free, or a leaked subtree, all of which are classic bugs in hand-written BST deletion code.

Moving Data Instead of Copying It

Course holds a std::vector<std::string> for its prerequisites, which means every copy of a Course also copies its entire prerequisite list. In Enhancement One, Insert() and addNode() both take Course by value and pass it down through each level of recursion, so inserting a course with several prerequisites into a deep tree copies that vector once per level of recursion. Enhancement Two passes course data through Insert() and addNode() using std::move, so the Course is constructed once and then transferred, not duplicated, into its final resting place in the tree. For a single insert this is a modest efficiency gain, but it reflects a correct understanding of what the recursive insertion algorithm is actually doing to the data it carries at each step, which is a meaningful distinction where algorithms and data structures are the primary focus..

A Unified Update Path Built on Search

Enhancement Two also adds an update capability that Enhancement One does not have. Rather than treating update as a separate, disconnected feature, its menu option first calls the tree's existing Search() to determine whether a courseId is already present. If it is, the new data is routed to updateCourse(), which reuses the same left/right comparison logic as Search() to walk down to the matching node and overwrite its title and prerequisites without disturbing the tree's shape. If the courseId is not found, the same input is routed to Insert() instead. This means the higher level update-or-insert behavior is built directly on top of the tree's core search algorithm rather than duplicating traversal logic in a separate code path, which keeps the number of independent ways the tree can be traversed and modified small and consistent.

Summary Comparison

Aspect	Enhancement One	Enhancement Two
Node ownership	Node uses raw Node* left/right pointers; the BST manually recurses through deleteBST() in the destructor to free every node.	Node uses std::unique_ptr<Node> left/right; the tree deallocates itself automatically through RAII. No destructor or deleteBST() is needed at all.
Delete operation	removeNode() is declared in the class but has no implementation; deleting a course is not possible.	removeNode() is fully implemented using the classic recursive BST deletion algorithm,

Aspect	Enhancement One	Enhancement Two
		correctly handling the no-child, one-child, and two-child cases via an in-order successor.
Duplicate keys	addNode() only compares '<' and falls into the right subtree for any non-smaller key, so inserting a courseId that already exists creates a second, structurally invalid node instead of updating the original.	addNode() checks for an exact courseId match before descending and overwrites the existing node's data, preserving the BST invariant that every key appears once.
Key normalization	courseId is stored exactly as read from the CSV or user input, so 'CS300' and 'cs300' would sort and compare as different keys.	Every courseId is passed through ToUpper() before insertion, search, update, or deletion, guaranteeing one consistent ordering key for the tree's comparisons.
Update capability	No update path exists; changing a course means there is no supported operation.	A dedicated updateCourse() traversal locates the node by courseId using the same comparison logic as Search(), then overwrites its title and

Aspect	Enhancement One	Enhancement Two
		prerequisites in place, without altering the tree's shape.
Insert efficiency	Course objects, which contain a std::vector of strings, are passed and reassigned by value through every level of recursion in addNode(), copying the vector at each step.	Course objects are passed with std::move throughout Insert() and addNode(), so a course is transferred into its final node rather than copied at every level of the recursive descent.
Traversal correctness under duplicates	Because duplicates are not detected, InOrder() can print two entries for the same course, and Search() will only ever find whichever one the traversal reaches first.	Because addNode() intercepts duplicates before insertion, InOrder() and Search() are guaranteed to reflect exactly one node per courseId.

What This Enhancement Demonstrates

Enhancement Two's changes are concentrated almost entirely inside the data structure rather than around it, and each one addresses a genuine algorithmic or structural gap: a missing deletion algorithm is implemented using the standard three-case recursive approach with an in-order successor; a broken key-uniqueness invariant is repaired at the point of insertion; a hidden key-

normalization bug that could split one logical course into two tree positions is closed; manual, error-prone memory management is replaced with ownership semantics the language itself enforces; and the insertion algorithm is changed to move data rather than copy it at every level of recursion. None of these are cosmetic, as each one is the kind of defect or missing capability that becomes visible only when a binary search tree is exercised the way one would actually be exercised in practice, which would be repeated insertions of the same key, deletions that require restructuring, and searches whose correctness depends on every node in the tree obeying the same ordering rule. Addressing them is what demonstrates a working understanding of how a BST is supposed to behave, not just how to write code that compiles and runs on the easy cases.

Even with these enhancements, one limitation is still open and worth noting honestly. The tree remains an unbalanced BST, so its search, insert, and delete operations run in O(h) time where h is the tree's height, and in the worst case, which would be courses loaded in already sorted order, that height can degrade toward O(n). Enhancement Two does not add self-balancing (for example, an AVL or red-black rebalancing step), so that remains a natural next step beyond this revision rather than something this enhancement claims to solve. However, Enhancement Three will solve this issue through the integration of a database management system.